

Selim Enes DÜNDAR

220104004036

CSE 241 OBJECT ORIENTED PROGRAMMING

HW1

```
8 // Function to remove comments from a line (everything after ';')
9 string removeComments(string line) {
10     int pos = line.find(';');
11     if (pos != string::npos) {
12         line = line.substr(0, pos);
13     }
14     return line;
15 }
16
17 // Function to remove whitespace from both ends of a string
18 string removeWhiteSpaces(string str) {
19     int start = 0;
20     int end = str.length();
21
22     while (start < end && (str[start] == ' ')) {
23         start++;
24     }
25     while (end > start && (str[end-1] == ' ')) {
26         end--;
27     }
28
29     return str.substr(start, end - start);
30 }
```

removeComments(string line)

Removes everything after the semicolon (;) character from a line, effectively stripping out comments.

removeWhiteSpaces(string str)

Removes leading and trailing whitespace characters from both ends of a string.

```

32 // Function to read all instructions from file
33 vector<string> readInstructions(string filename) {
34     vector<string> instructions;
35     ifstream file(filename.c_str());
36
37     if (!file.is_open()) {
38         cout << "Error: Cannot open file " << filename << endl;
39         return instructions;
40     }
41
42     string line;
43     while (getline(file, line)) {
44         instructions.push_back(line);
45     }
46
47     file.close();
48     return instructions;
49 }
50

```

I opened the file, if it fails to open I printed an error message and returned an empty vector. If it opened successfully, I read each line and added it to the vector, then closed the file and returned the vector.

```

200 int executeJMP(vector<string>& parts, int registers[], int currentLine) {
201     // JMP lineAddress (unconditional)
202     if (parts.size() == 2) {
203         if (isNumber(parts[1])) {
204             return stoi(parts[1]) - 1; // -1 because lines are 1-indexed
205         } else {
206             cout << "Error: Invalid line address" << endl;
207             return -1;
208         }
209     }
210     // JMP reg, lineAddress (conditional)
211     else if (parts.size() == 3) {
212         int reg = getRegisterIndex(parts[1]);
213         if (reg == -1) {
214             cout << "Error: Invalid register" << endl;
215             return -1;
216         }
217
218         if (isNumber(parts[2])) {
219             if (registers[reg] == 0) {
220                 return stoi(parts[2]) - 1; // Jump if register is zero
221             } else {
222                 return currentLine + 1; // Continue to next line
223             }
224         } else {
225             cout << "Error: Invalid line address" << endl;
226             return -1;
227         }
228     }

```

executeJMP(vector<string>& parts, int registers[], int currentLine) Function

I handled unconditional jump (2 parts) by returning the line number minus 1 if valid, otherwise returned -1 with an error.

For conditional jump (3 parts), I got the register index, checked if it's valid, then returned the target line minus 1 if the register equals 0, otherwise returned the next line. If anything was invalid, I returned -1 with an error.

```

276 void executeProgram(vector<string>& instructions, int option) {
277     int registers[6] = {0, 0, 0, 0, 0, 0}; // Initialize all 6 registers to 0
278     int program_counter = 0; // Program counter (current line)
279
280     while (program_counter < (int)instructions.size()) {
281         string line = instructions[program_counter];
282         vector<string> parts = parseInstruction(line);
283
284         // Skip empty lines
285         if (parts.empty()) {
286             program_counter++;
287             continue;
288         }
289
290         string instruction = parts[0];
291
292         // Option 1: Print instruction before execution
293         if (option == 1) {
294             cout << removeWhiteSpaces(removeComments(line)) << " - ";
295         }
296
297         // Execute the instruction
298         bool success = true;
299         int nextPC = program_counter + 1; // Default: move to next line
300

```

executeProgram(vector<string>& instructions, int option) Function

I initialized 6 registers to 0 and set the program counter to 0.

I processed all instructions with a while loop. I parsed each line with parseInstruction.

I skipped empty lines (incremented counter and continued).

If option is 1, I printed the instruction after removing comments and whitespaces.

I created a success flag and nextPC variable (default: next line).

```

if (instruction == "MOV") {
    success = executeMOV(parts, registers);
}
else if (instruction == "ADD") {
    success = executeADD(parts, registers);
}
else if (instruction == "SUB") {
    success = executeSUB(parts, registers);
}
else if (instruction == "JMP") {
    nextPC = executeJMP(parts, registers, program_counter);
    if (nextPC == -1) success = false;
}
else if (instruction == "PRN") {
    success = executePRN(parts, registers);
}
else if (instruction == "INP") {
    success = executeINP(parts, registers);
}
else if (instruction == "HLT") {
    if (option == 1) {
        printRegisters(registers);
    }
}

```

checked the instruction type with if-else statements:

If instruction is "MOV": I called executeMOV with parts and registers, and stored the result in success.

If instruction is "ADD": I called executeADD with parts and registers, and stored the result in success.

If instruction is "SUB": I called executeSUB with parts and registers, and stored the result in success.

If instruction is "JMP": I called executeJMP with parts, registers, and program_counter, and stored the returned value in nextPC. If nextPC equals -1, I set success to false.

If instruction is "PRN": I called executePRN with parts and registers, and stored the result in success.

If instruction is "INP": I called executeINP with parts and registers, and stored the result in success.

If instruction is "HLT": If option equals 1, I called printRegisters to display the registers.

```

351  ✓ int main(int argc, char* argv[]) {
352      // Check command line arguments
353  ✓  if (argc != 3) {
354      cout << "Usage: " << argv[0] << " <filename> <option>" << endl;
355      cout << "option = 0: Run silently" << endl;
356      cout << "option = 1: Run with instruction display" << endl;
357      return 1;
358  }
359
360      string filename = argv[1];
361      int option = stoi(argv[2]);
362
363      // Validate option
364  ✓  if (option != 0 && option != 1) {
365      cout << "Error: Option must be 0 or 1" << endl;
366      return 1;
367  }
368
369      // Read instructions from file
370      vector<string> instructions = readInstructions(filename);
371
372  ✓  if (instructions.empty()) {
373      cout << "Error: No instructions to execute" << endl;
374      return 1;
375  }
376
377      // Execute the program
378      executeProgram(instructions, option);

```

main Function

Check arguments: If not exactly 3 arguments, print usage and exit.

Get filename and option: Read from argv[1] and argv[2].

Validate option: Option must be 0 or 1, otherwise error.

Read file: Load instructions from file into vector.

Check if empty: If no instructions, print error and exit.

Execute: Run the program with executeProgram(instructions, option).

OUTPUTS

Sum of squares option = 0

```
PS C:\Users\phoni\OneDrive\Masaüstü\oop hw1> .\a.exe .\sum_of_squares.txt 0
385
Program halted.
R1=385, R2=100, R3=0, R4=0, R5=0, R6=0
PS C:\Users\phoni\OneDrive\Masaüstü\oop hw1> █
```

Sum of squares option = 1

```
PS C:\Users\phoni\OneDrive\Masaüstü\oop hw1> .\a.exe .\sum_of_squares.txt 1
MOV R1, 0 - R1=0, R2=0, R3=0, R4=0, R5=0, R6=0
MOV R2, 1 - R1=0, R2=1, R3=0, R4=0, R5=0, R6=0
ADD R1, R2 - R1=1, R2=1, R3=0, R4=0, R5=0, R6=0
MOV R2, 4 - R1=1, R2=4, R3=0, R4=0, R5=0, R6=0
ADD R1, R2 - R1=5, R2=4, R3=0, R4=0, R5=0, R6=0
MOV R2, 9 - R1=5, R2=9, R3=0, R4=0, R5=0, R6=0
ADD R1, R2 - R1=14, R2=9, R3=0, R4=0, R5=0, R6=0
MOV R2, 16 - R1=14, R2=16, R3=0, R4=0, R5=0, R6=0
ADD R1, R2 - R1=30, R2=16, R3=0, R4=0, R5=0, R6=0
MOV R2, 25 - R1=30, R2=25, R3=0, R4=0, R5=0, R6=0
ADD R1, R2 - R1=55, R2=25, R3=0, R4=0, R5=0, R6=0
MOV R2, 36 - R1=55, R2=36, R3=0, R4=0, R5=0, R6=0
ADD R1, R2 - R1=91, R2=36, R3=0, R4=0, R5=0, R6=0
MOV R2, 49 - R1=91, R2=49, R3=0, R4=0, R5=0, R6=0
ADD R1, R2 - R1=140, R2=49, R3=0, R4=0, R5=0, R6=0
MOV R2, 64 - R1=140, R2=64, R3=0, R4=0, R5=0, R6=0
ADD R1, R2 - R1=204, R2=64, R3=0, R4=0, R5=0, R6=0
MOV R2, 81 - R1=204, R2=81, R3=0, R4=0, R5=0, R6=0
ADD R1, R2 - R1=285, R2=81, R3=0, R4=0, R5=0, R6=0
MOV R2, 100 - R1=285, R2=100, R3=0, R4=0, R5=0, R6=0
ADD R1, R2 - R1=385, R2=100, R3=0, R4=0, R5=0, R6=0
PRN R1 - 385
R1=385, R2=100, R3=0, R4=0, R5=0, R6=0
HLT - R1=385, R2=100, R3=0, R4=0, R5=0, R6=0
Program halted.
R1=385, R2=100, R3=0, R4=0, R5=0, R6=0
```

Countdown option = 1


```

PS C:\Users\phoni\OneDrive\Masaüstü\oop_hw1> .\a.exe .\countdown.txt 1
MOV R1, 10 - R1=10, R2=0, R3=0, R4=0, R5=0, R6=0
PRN R1 - 10
R1=10, R2=0, R3=0, R4=0, R5=0, R6=0
SUB R1, 1 - R1=9, R2=0, R3=0, R4=0, R5=0, R6=0
JMP R1, 6 - R1=9, R2=0, R3=0, R4=0, R5=0, R6=0
JMP 2 - R1=9, R2=0, R3=0, R4=0, R5=0, R6=0
PRN R1 - 9
R1=9, R2=0, R3=0, R4=0, R5=0, R6=0
SUB R1, 1 - R1=8, R2=0, R3=0, R4=0, R5=0, R6=0
JMP R1, 6 - R1=8, R2=0, R3=0, R4=0, R5=0, R6=0
JMP 2 - R1=8, R2=0, R3=0, R4=0, R5=0, R6=0
PRN R1 - 8
R1=8, R2=0, R3=0, R4=0, R5=0, R6=0
SUB R1, 1 - R1=7, R2=0, R3=0, R4=0, R5=0, R6=0
JMP R1, 6 - R1=7, R2=0, R3=0, R4=0, R5=0, R6=0
JMP 2 - R1=7, R2=0, R3=0, R4=0, R5=0, R6=0
PRN R1 - 7
R1=7, R2=0, R3=0, R4=0, R5=0, R6=0
SUB R1, 1 - R1=6, R2=0, R3=0, R4=0, R5=0, R6=0
JMP R1, 6 - R1=6, R2=0, R3=0, R4=0, R5=0, R6=0
JMP 2 - R1=6, R2=0, R3=0, R4=0, R5=0, R6=0
PRN R1 - 6
R1=6, R2=0, R3=0, R4=0, R5=0, R6=0
SUB R1, 1 - R1=5, R2=0, R3=0, R4=0, R5=0, R6=0
JMP R1, 6 - R1=5, R2=0, R3=0, R4=0, R5=0, R6=0
JMP 2 - R1=5, R2=0, R3=0, R4=0, R5=0, R6=0
PRN R1 - 5
R1=5, R2=0, R3=0, R4=0, R5=0, R6=0
SUB R1, 1 - R1=4, R2=0, R3=0, R4=0, R5=0, R6=0
JMP R1, 6 - R1=4, R2=0, R3=0, R4=0, R5=0, R6=0
JMP 2 - R1=4, R2=0, R3=0, R4=0, R5=0, R6=0

```

```

PRN R1 - 4
R1=4, R2=0, R3=0, R4=0, R5=0, R6=0
SUB R1, 1 - R1=3, R2=0, R3=0, R4=0, R5=0, R6=0
JMP R1, 6 - R1=3, R2=0, R3=0, R4=0, R5=0, R6=0
JMP 2 - R1=3, R2=0, R3=0, R4=0, R5=0, R6=0
PRN R1 - 3
R1=3, R2=0, R3=0, R4=0, R5=0, R6=0
SUB R1, 1 - R1=2, R2=0, R3=0, R4=0, R5=0, R6=0
JMP R1, 6 - R1=2, R2=0, R3=0, R4=0, R5=0, R6=0
JMP 2 - R1=2, R2=0, R3=0, R4=0, R5=0, R6=0
PRN R1 - 2
R1=2, R2=0, R3=0, R4=0, R5=0, R6=0
SUB R1, 1 - R1=1, R2=0, R3=0, R4=0, R5=0, R6=0
JMP R1, 6 - R1=1, R2=0, R3=0, R4=0, R5=0, R6=0
JMP 2 - R1=1, R2=0, R3=0, R4=0, R5=0, R6=0
PRN R1 - 1
R1=1, R2=0, R3=0, R4=0, R5=0, R6=0
SUB R1, 1 - R1=0, R2=0, R3=0, R4=0, R5=0, R6=0
JMP R1, 6 - R1=0, R2=0, R3=0, R4=0, R5=0, R6=0
HLT - R1=0, R2=0, R3=0, R4=0, R5=0, R6=0
Program halted.
R1=0, R2=0, R3=0, R4=0, R5=0, R6=0
PS C:\Users\phoni\OneDrive\Masaüstü\oop_hw1>

```

Countdown option = 0

```
PS C:\Users\phoni\OneDrive\Masaüstü\oop hw1> .\a.exe .\countdown.txt 0
10
9
8
7
6
5
4
3
2
1
Program halted.
R1=0, R2=0, R3=0, R4=0, R5=0, R6=0
```